



PowerShell PCI Tool (No Driver)

PCI EXPRESS INVENTORY & LINK DIAGNOSTICS

winlspci User's Manual

`lspci` for Windows — without a kernel driver

A PowerShell tool that enumerates every PCI and PCI Express device in a Windows machine, reports negotiated *and* maximum link speed and width, draws the real bus topology, and answers questions at the attribute level — with no driver to sign, install, or trust.

```
PS> lspci -nn
00:1c.0 PCI bridge [0604]: Intel Corporation Tigerlake PCH-LP PCI
Express Root Port #6 [8086:a0bd] (rev 20)
01:00.0 Non-Volatile memory controller [0108]: SK hynix Gold
P31/BC711/PC711 NVMe SSD [1c5c:174a] (rev 00)
```

Document revision 1.2 · August 2026 · Requires Windows PowerShell 5.1 · MIT Licence
github.com/jpmutschler/winlspci

Contents

1	About winlspci	2
2	What winlspci can and cannot report	3
3	Requirements and installation	4
4	Quick start	6
5	Command reference — standard lspci flags	7
6	Command reference — flags beyond lspci	9
7	Attribute-level queries	10
8	Delimited output and coming from Linux	11
9	Topology	12
10	PCI domains (segments)	13
11	The PCI ID database	14
12	Design principles	15
13	Performance	17
14	Exit codes and scripting	18
15	Troubleshooting	19
16	Tests	21
A	Appendix A — Flag quick reference	21
B	Appendix B — Repository layout	23
C	Appendix C — Glossary	24
D	Appendix D — Licence and credits	25

A NOTE ON NAMES

winlspci is the name of the tool. The command you type is `lspci`, deliberately, so that muscle memory and existing notes carry over from Linux. Where this manual writes `lspci` in an example, that is the literal command; where it writes `winlspci`, it means the tool as a whole — the module, its cmdlets and its data files.

[NO KERNEL DRIVER](#)[WINDOWS POWERSHELL 5.1](#)[OFFLINE PCI.IDS](#)[REAL TOPOLOGY](#)[TYPED OBJECTS](#)[JSON / CSV](#)[MIT](#)

1 About winlspci

winlspci answers the question every bench engineer asks first: *what is actually in this machine, and is it running the way it should be?* It does that on Windows, from a normal PowerShell prompt, with nothing installed into the kernel.

1.1 What it is

winlspci is a Windows PowerShell implementation of the familiar Linux `lspci` utility. It lists PCI and PCI Express devices with their vendor, device, subsystem and class identifiers, resolves those identifiers to human-readable names from a bundled copy of the PCI ID database, reports link speed and width — both what the link negotiated and what it is capable of — and can draw the bus as a real parent/child tree.

Beyond that, it does something `lspci` does not: it exposes every field it knows as a typed, queryable attribute, so you can ask precise questions without piping text through a search tool that Windows does not have.

Source and issues: <https://github.com/jpmutschler/winlspci> · MIT · Windows PowerShell 5.1

1.2 Why there is no driver

This is the most important thing to understand about the tool, and the reason its capabilities are shaped the way they are.

THE CONSTRAINT

Windows exposes no userland path to PCI configuration space. A faithful port of `lspci` is therefore impossible without a signed kernel-mode driver — and shipping one of those means driver signing, Secure Boot friction, and a considerably larger attack surface on a lab machine.

Rather than accept that cost, winlspci reads Windows' own PnP/PCI enumeration data instead. That data is populated by the Windows PCI bus driver, which reads configuration space itself at enumeration time. The consequence is a clean trade: everything the bus driver chose to surface is available instantly and safely; everything it did not surface is genuinely out of reach, and winlspci says so rather than guessing.

1.3 Design philosophy

One idea runs through the whole tool, and it is worth stating before the reference chapters:

THE GOVERNING RULE

A wrong answer that looks right is worse than an obvious failure.

That principle is why a value the hardware did not report prints as *not reported* rather than as zero; why a filter that matches nothing returns a non-zero exit code instead of an empty-but-successful result; and why asking for a configuration-space dump produces a clear refusal instead of a partial answer dressed up as a complete one. Chapter 12 covers each of these in detail.

1.4 Who it is for

Hardware validation

Confirm a card enumerated at all, at the right slot, with the right IDs — then confirm the link trained to the speed and width the design calls for.

Field and support

Capture a complete, structured inventory of an unfamiliar machine to CSV or JSON with one command, with no software to install on a customer's system.

Linux engineers on Windows

Keep the flags, the output shape and the reflexes you already have, without pretending Windows has `grep`, `awk` or `cut`.

2 What winlspci can and cannot report

Read this chapter before the reference chapters. The boundary below is not a list of features that have not been written yet — it is the boundary of what any driverless tool on Windows can see.

REPORTED	CANNOT BE REPORTED
Device presence; vendor, device and subsystem IDs; revision	Configuration-space hex dumps (<code>-x</code>)
Class, subclass and prog-if, with resolved names	Capability structure walks
<code>[domain:]bus:device.function</code> — including the PCI segment, where Windows reports one (Hyper-V / Azure)	ASPM state, LTR, DPC
Negotiated and maximum link speed and link width	AER register detail (presence only is reported)
Max Payload Size (MPS) and Max Read Request size (MRRS)	Anything requiring a live register read
Driver binding and driver version; device status and problem code	
NUMA node; AER capability presence	
Bus topology (parent/child, via <code>DEVPKEY_Device_Parent</code>)	
Every field above, queryable per attribute	

2.1 What happens when you ask for the impossible

winlspci does not silently do less than you asked. A request that falls in the right-hand column above produces an explanation and a non-zero exit code:

```
PS> lspci -x
lspci: cannot dump configuration space (-x). Windows exposes no userland path
to PCI config space; that needs a signed kernel-mode driver. ...
```

The command exits with status **2**. A script that assumed `-x` would work therefore fails loudly at the point of the mistaken assumption, rather than quietly producing an inventory with a hole in it. The same exit code covers known lspci flags this tool recognises but has not implemented — see §5.4 and chapter 14.

PRACTICAL READING

If your task is *presence, identity, binding, link state and topology*, winlspci covers it completely. If your task requires reading a register on the device right now — a capability walk, an ASPM state, an AER error log — no driverless tool can help, and you need a kernel-mode utility from your silicon vendor.

3 Requirements and installation

There is no installer, no service, and no driver. Clone the repository and run it.

3.1 Requirements

Operating system	Windows, with the standard PnP/PCI enumeration available (that is, a normally configured system)
Shell	Windows PowerShell 5.1 — the version that ships with Windows. No separate runtime to install.
Network	Not required. The PCI ID database is bundled.
Kernel driver	None. That is the point of the tool.
Source and issues	https://github.com/jpmutschler/winlspci

WHY 5.1 SPECIFICALLY

winlspci is written to Windows PowerShell 5.1 syntax on purpose — no ternary operator, no `??` null-coalescing, no `-AsHashtable`. The reasoning is simple: **a tool that tells you what is in the machine should not need an install before it runs**. Any Windows box you sit down at can run it as-is.

3.2 Clone and run

```
PS> git clone https://github.com/jpmutschler/winlspci.git $env:LOCALAPPDATA\Programs\winlspci
PS> $env:PATH += ";$env:LOCALAPPDATA\Programs\winlspci\bin"
PS> lspci -nn
```

That gives you the `lspci` command for the current session only. To make it permanent, append to the **User** PATH:

```
PS> $p = [Environment]::GetEnvironmentVariable('PATH','User')
PS> [Environment]::SetEnvironmentVariable('PATH', "$p;$env:LOCALAPPDATA\Programs\winlspci\bin",
'User')
```

DO NOT USE SETX

Use the .NET API shown above, **not** `setx`. The `setx` command silently truncates PATH at 1024 characters, and a developer machine is routinely well past that length. Truncating PATH destroys most of your environment with no warning and no error message.

3.3 Why not `C:\tools`

A FOLDER UNDER THE DRIVE ROOT IS WRITABLE BY EVERYONE

A folder created directly under `C:\` inherits *Modify* for every authenticated user, so any standard account could rewrite `lspci.ps1` for the next administrator who runs it. A hardware-debug tool is exactly the thing that gets run from an elevated shell.

Per-user (`%LOCALAPPDATA%\Programs`) or `C:\Program Files` are both fine. The commands above use the former.

3.4 Importing the module directly

If you would rather work with the cmdlets than the `lspci` command wrapper, import the module and call them:

```
PS> Import-Module $env:LOCALAPPDATA\Programs\winlspci\winlspci.psd1
PS> Get-PciDevice -Device '10de:' | Format-Lspci -Verbosity 2
```

The cmdlets return real PowerShell objects, so everything downstream of them — `Where-Object`, `Select-Object`, `Export-Csv`, comparison operators on numeric fields — behaves the way you expect. Chapter 7 builds on this.

3.5 Verifying the installation

```
PS> lspci -nn           # should print one line per device
PS> lspci -Version      # tool version, and the date of the bundled pci.ids
PS> lspci -ListAttributes # should print the queryable attribute names
PS> (lspci).Count       # device count – a sanity check against Device Manager
```

4 Quick start

Five commands that cover most of what the tool is used for.

4.1 List everything, with names and IDs

```
PS> lspci -nn
00:1c.0 PCI bridge [0604]: Intel Corporation Tigerlake PCH-LP PCI Express Root Port #6
[8086:a0bd] (rev 20)
01:00.0 Non-Volatile memory controller [0108]: SK hynix Gold P31/BC711/PC711 NVMe SSD
[1c5c:174a] (rev 00)
```

4.2 Look closely at one vendor's devices

```
PS> lspci -d 1c5c: -vv
01:00.0 Non-Volatile memory controller: SK hynix Gold P31/BC711/PC711 NVMe Solid State Drive
(rev 00)
    LnkSta: 8GT/s x4 (max 8GT/s x4)
    Driver: stornvme (10.0.26100.8972)
    Subsystem: SK hynix [1c5c:174a]
    DevCtl: MPS 256 bytes (max 512), MaxReadReq 512 bytes
    Capabilities: AER present
```

Note the `LnkSta` line: the negotiated speed and width, and in parentheses the maximum the link is capable of. Those two numbers agreeing is the answer to most link questions; them disagreeing is the beginning of an investigation.

4.3 See the tree

```
PS> lspci -t
-00:1d.0 Tiger Lake-LP PCI Express Root Port #9
 \-f3:00.0 GA107M [GeForce RTX 3050 Ti Mobile] (8GT/s x4)
```

4.4 Find fast links — without grep

```
PS> lspci -Attribute LinkSpeed -Match '16GT|32GT|64GT' # no grep needed
```

4.5 Find anything running below its maximum

```
PS> lspci -Downtrained # anything below its max
```

INTERPRETING -DOWNTRAINED

A downtrained link is a *finding*, not automatically a fault. On a laptop, a GPU reporting 8GT/s x4 against a maximum of 16GT/s x16 is almost certainly link power management at idle. winlspci tells you what the link is doing; deciding whether that is a problem is still yours.

5 Command reference — standard lspci flags

Measured against Linux `lspci`, not against an aspiration. Three lists follow: what is implemented, what is refused on principle, and what is recognised but not built.

5.1 Implemented

LSPCI	WINLSPCI	NOTES
<code>-s [[<dom>]:]<bus>[:] <dev>[.<fn>]</code>	<code>-s</code>	<code>lspci</code> 's grammar, with every field optional: <code>01:</code> (bus), <code>01:00.0</code> , <code>1</code> (device 01 on any bus), <code>.0</code> (every function 0), <code>:00.0</code> , <code>0000:01:00.0</code> . Padded or unpadded. A selector <i>without</i> a domain matches every domain; <code>556f:00:02.0</code> matches only that one
<code>-d [<ven>]: [<dev>] [: <class>]</code>	<code>-d</code>	Vendor and device are exact hex IDs — <code>-d 80:</code> is vendor <code>0080</code> , not every <code>80xx</code> . Class is a <i>prefix</i> : <code>::01</code> is all storage. Examples: <code>-d 11f8:</code> , <code>-d :174a</code> , <code>-d ::0108</code>
<code>-t</code>	<code>-t</code>	Real topology, built from <code>DEVPKEY_Device_Parent</code> . Shows link state inline
<code>-v</code>	<code>-v</code>	Link state (current and maximum), driver, device status
<code>-vv</code>	<code>-vv</code>	Adds MPS, MRRS, subsystem (as <code>vendor:device</code>), NUMA node and AER presence
<code>-vvv</code>	<code>-vvv</code>	Everything <code>-vv</code> shows, then every property Windows exposes, plus an explicit list of what is missing
<code>-n</code>	<code>-n</code>	Numeric IDs only
<code>-nn</code>	<code>-nn</code>	Names and IDs together
<code>-D</code>	<code>-D</code>	Domain on every line. <code>0000</code> on ordinary machines. Hyper-V and Azure SR-IOV functions carry the PCI segment in the upper bits of Windows' bus number (<code>PCI bus 5598976</code> = segment <code>556f</code> , bus <code>00</code>); those always show it, as <code>lspci</code> does: <code>556f:00:02.0</code> . See chapter 10
<code>-k</code>	<code>-k</code>	Driver and version — the same lines <code>-v</code> shows
<code>-tv</code> , <code>-nnk</code> , <code>-vvnn</code> ...	same	Short flags combine, as in <code>lspci</code> . They are case-sensitive : <code>-D</code> $\neq$ <code>-d</code>

SHORT FLAGS VERSUS LONG OPTIONS

Short lspci flags are case-sensitive, because `-D` and `-d` are two different things in lspci and must stay that way here. The long options that are unique to this tool — `-Json`, `-Downtrained`, `-Attribute` and the rest — are case-*ins*ensitive and may be abbreviated to any unique prefix.

5.2 Values and the PowerShell parser

ONLY RELEVANT IF YOU BYPASS THE LAUNCHER

Run through the `lspci` launcher (`bin\lspci.cmd`, the one on PATH) and every form works exactly as typed, because the script recovers the raw command line.

If you call `bin\lspci.ps1` directly from a PowerShell *prompt*, PowerShell's own parser gets there first: it splits `-s01:` at the colon, and reads a bare `.0` or `00.0` as the number `0`. At the prompt, put a space before the value and quote anything beginning with a dot or a colon.

```
PS> .\lspci.ps1 -s '.0'
PS> .\lspci.ps1 -s ':00.0'
PS> .\lspci.ps1 -s 01:00.0
```

5.3 Deliberately refused

LSPCI	WHY
<code>-x</code> , <code>-xxx</code> , <code>-xxxx</code>	Configuration-space dumps. Impossible without a signed kernel-mode driver. winlspci exits 2 with an explanation rather than pretending otherwise.

5.4 Not implemented (possible, just not built)

These are absent by omission rather than by principle. Nothing about the driverless approach prevents them.

FLAG	PURPOSE
<code>-b</code>	Bus-centric view
<code>-m</code> , <code>-mm</code>	Machine-readable output — superseded here by <code>-Json</code> , <code>-Csv</code> and <code>-Attribute</code>
<code>-p</code>	Custom ID file
<code>-P</code>	Path display
<code>-i</code>	Custom <code>pci.ids</code> path
<code>-q</code> , <code>-Q</code>	Online ID lookup
<code>-A</code> , <code>-0</code> , <code>-F</code> , <code>-G</code> , <code>-H1</code> , <code>-H2</code>	Access-method selection — meaningless in a driverless implementation
<code>-M</code>	Bus mapping

Each of these is **recognised**. Passing one exits 2 with “*not implemented*” and a pointer to the nearest equivalent. Anything else unknown exits **64**.

WHY RECOGNISE A FLAG YOU DO NOT SUPPORT

An earlier version let PowerShell's parameter binder guess, and the results were quietly wrong: `-p ran -PresentOnly`, and `-m` demanded a value for `-Match`. **A tool that runs a different command from the one you typed is worse than one that says no.**

6 Command reference — flags beyond lspci

These flags have no Linux counterpart. They exist because PowerShell's object pipeline makes possible what a text pipeline cannot do safely. All of them are case-insensitive and may be abbreviated to a unique prefix.

FLAG	WHY IT EXISTS
<code>-Attribute <names></code>	Query at the attribute level rather than the device level. Wildcards are accepted: <code>-Attribute Link*</code>
<code>-Match <regex></code>	Filter by value — the job you would otherwise have handed to <code>grep</code>
<code>-PresentOnly</code>	Drop attributes the device does not report, leaving only real data
<code>-ListAttributes</code>	Discover what is queryable on this build. Answered from the module's static list, so it does not enumerate the machine
<code>-Delimited</code>	<code> </code> -separated records, for <code>-split</code> / <code>cut</code> / <code>awk</code> habits
<code>-Delimiter</code>	Change the separator — for example a tab, giving TSV
<code>-Header</code>	Name the columns in delimited output
<code>-Json</code>	Structured JSON output
<code>-Csv</code>	Structured CSV output, correctly quoted
<code>-Downtrained</code>	Only devices below their maximum speed or width — the <code>Downtrained</code> property. This is a <i>filter</i> : it exits 1 when nothing is downtrained
<code>-Version</code>	Tool version, and the date of the bundled <code>pci.ids</code>

CHOOSING BETWEEN THEM

Reach for `-Attribute` and `-Match` when you want a specific answer, `-Csv` or `-Json` when something else will consume the output, and `-Delimited` when you already know the one-liner you want to write.

7 Attribute-level queries

Windows has no `grep`, so `lspci -vv | grep LnkSta` has no equivalent. Rather than bolt a text search onto the tool, winlspci serialises the data to one record per attribute — which turns out to be strictly better than the thing it replaces.

7.1 One record per attribute

```
PS> lspci -Attribute Link* -s 01:00.0 -PresentOnly
```

Slot	VendorId	DeviceId	Attribute	Value	Present
01:00.0	1c5c	174a	LinkStateReported	True	True
01:00.0	1c5c	174a	LinkSpeed	8GT/s	True
01:00.0	1c5c	174a	LinkWidth	4	True

7.2 Filtering by value

This is the part you would have piped to `grep`:

```
PS> lspci -Attribute LinkSpeed -Match '16GT|32GT|64GT' # every Gen4+ link
PS> lspci -Attribute Driver -Match 'stornvme'          # everything on one driver
PS> lspci -Attribute '*' -Match '11f8' -Csv           # every field mentioning the vendor
PS> lspci -ListAttributes                             # what can I ask for?
```

7.3 Three properties that beat text search

Present is a field

Absent and zero never collapse into each other. A root port that reports no link width is not a device running at x0 — and a text pipeline cannot tell those apart.

Values keep their types

`LinkWidth` is an integer, so `-lt 4` is a numeric comparison. `grep` would have you comparing strings.

No match exits 1

An attribute query that matches nothing returns a non-zero exit code, so a script can tell “no match” from “it worked”.

7.4 Composing with the pipeline

Because these are real objects, ordinary PowerShell applies:

```
# every device not running at full width, as a table
Get-PciDevice | Where-Object { $_.LinkStateReported -and $_.LinkWidth -lt $_.MaxLinkWidth } |
    Select-Object Slot, DeviceName, LinkWidth, MaxLinkWidth

# diff two machines
Get-PciDevice | ConvertTo-PciAttributeRecord | Export-Csv machine-a.csv
```

Exporting the attribute records from two systems and diffing the two CSV files is the fastest way to answer “what is different about this machine?” — a question that is otherwise a long afternoon.

8 Delimited output and coming from Linux

You cannot have `grep`, `awk` and `cut` here, but you can have the shape of them.

8.1 Delimited records

`-Delimited` emits one record per line with `|` between fields:

```
PS> lspci -Delimited -s 01:
01:00.0|1c5c|174a|0108|Non-Volatile memory controller|SK hynix|Gold P31 NVMe
SSD|00|86T/s|4|86T/s|4|stornvme|OK
```

`-Header` names the columns; `-Delimiter "`t"` gives you TSV.

8.2 Translation table

LINUX	WINLSPCI
<code>lspci grep NVMe</code>	<code>lspci Select-String NVMe</code>
<code>lspci -vv grep LnkSta</code>	<code>lspci -Attribute LinkSpeed,LinkWidth</code>
<code>lspci cut -d' ' -f1</code>	<code>lspci -Delimited %{ (\$_ -split '\ ')[0] }</code>
<code>lspci awk -F' ' '{print \$1, \$9}'</code>	<code>lspci -Delimited %{ \$f=\$_ -split '\ '; "\$(\$f[0]) \$(\$f[8])" }</code>
<code>lspci wc -l</code>	<code>(lspci).Count</code>
<code>lspci -d 11f8: echo "absent"</code>	Same — a filter matching nothing exits 1

8.3 Two properties worth relying on

An empty field means “not reported”; a literal `0` means zero

The distinction the object model protects survives into the text form for free, because an unset value serialises to nothing at all. This is real output from the machine `winlspci` was written on:

```
00:1e.0|8086|a0a8|0780|Communication controller|Intel Corporation|Tiger Lake-LP Serial IO UART
Controller #0|20|||||iaLPSS2_UART2_TGL|OK
    the four consecutive empty fields above are the link-state columns: not reported
f3:00.0|10de|25a0|0302|3D controller|NVIDIA Corporation|GA107M [GeForce RTX 3050 Ti
Mobile]|a1|86T/s|4|166T/s|16|nvlddmkm|OK
```

A device with no link state and a device genuinely training at x0 are different findings, and `grep`-shaped output normally destroys that difference. Here it survives a `-split`.

THE SECOND LINE, READ CLOSELY

That GPU reports **8GT/s x4 against a maximum of 16GT/s x16** — which is exactly why `-Downtrained` exists. On a laptop it is almost certainly link power management at idle rather than a fault, and that is the point: the tool tells you what the link is doing, and the judgement stays with you.

The delimiter is stripped from values, not escaped

Quoting rules turn a one-liner into a parser, which defeats the purpose — so a `|` appearing inside a value becomes `/` and the column count stays fixed. No `|` occurs in live PCI data or in `pci.ids` today, but `FriendlyName` comes from driver INF files and is not under the tool's control.

IF YOU NEED REAL QUOTING

Use `-Csv`. It quotes properly. `-Delimited` is a convenience for known-shaped pipelines, not a general-purpose serialisation format.

STRUCTURED BEATS TEXTUAL WHERE IT CAN

`lspci -Attribute LinkSpeed -Match '16GT|32GT|64GT' keepstypes`, so `LinkWidth -lt 4` is a numeric comparison rather than a string one. Reach for `-Delimited` when you already know the pipeline you want to write.

9 Topology

`-t` builds a real tree from `DEVPKEY_Device_Parent`, so an endpoint appears under the root port that actually carries it — with its link state inline, which is where a downtrained device becomes obvious at a glance.

```
-00:06.0 11th Gen Core Processor PCIe Controller
 \-01:00.0 Gold P31/BC711/PC711 NVMe Solid State Drive (8GT/s x4)
-00:1d.0 Tiger Lake-LP PCI Express Root Port #9
 \-f3:00.0 GA107M [GeForce RTX 3050 Ti Mobile] (8GT/s x4)
```

9.1 Reading the tree

- A line beginning `-` at the left margin is a root of the tree — typically a root port or host bridge.
- `\-` marks a child of the entry above it.
- The value in parentheses is the endpoint's current link state, in *speed × width* form.

9.2 Devices with a non-PCI parent

A device whose parent is not itself a PCI device becomes a root of the tree rather than being dropped from it.

WHY THAT CHOICE

An incomplete tree that *looks* complete is worse than an obviously ragged one. A test asserts that every enumerated device appears in the tree exactly once, so what you see is the whole inventory even when the hierarchy is untidy.

10 PCI domains (segments)

On an ordinary machine every device is in domain `0000`, the slot is `bus:device.function`, and `-D` is the only way to see the domain — exactly as in `lspci`. Virtualised hardware is the exception, and it is worth understanding before you meet it.

10.1 Where the segment hides on Hyper-V and Azure

Hyper-V and Azure report SR-IOV functions on a “bus” that is really segment and bus packed together.

Windows says `PCI bus 5598976`, which is `0x556F00` — and since a PCI bus number is only 8 bits wide, the upper bits are the segment.

HOW THIS WAS FOUND

An earlier version printed that value as `bus 556f00`: a slot nothing could parse. The first CI run on a GitHub Windows runner — an Azure VM with one Mellanox ConnectX-4 virtual function — found it. Hardware nobody owns is exactly the hardware a test machine will hand you.

The bus number is now split into a `Domain` property (the upper 16 bits) and the bus itself (the low 8). As `lspci` does, a non-zero domain is always shown in the slot:

```
PS> lspci -nn                                # on an Azure VM
3851:00:00.0 Non-Volatile memory controller [0108]: Microsoft Corporation Standard NVM Express
Controller [1414:b111] (rev 01)
7870:00:00.0 Ethernet controller [0200]: Microsoft Corporation Microsoft Azure Network Adapter
Virtual Bus [1414:00ba] (rev 00)
c05b:00:00.0 Non-Volatile memory controller [0108]: Microsoft Corporation ASAP NVM Express
Controller [1414:00a9] (rev 00)
```

10.2 The rules, in full

Slot format	<code>[dddd:]bb:dd.f</code> — the domain appears only when there is one
The <code>Domain</code> property	An integer (<code>0</code> on an ordinary machine), and one of the queryable attributes: <code>lspci -Attribute Domain</code>
Selecting without a domain	<code>-s</code> without a domain matches <i>every</i> domain — <code>-s 00:00.0</code> finds all three devices above
Selecting one domain	<code>-s 7870:00:00.0</code> finds exactly one. A domain nobody is in matches nothing and exits 1
What <code>-D</code> does	Prefixes <code>0000:</code> only where the slot has no domain already. It never produces <code>0000:7870:...</code>
Sort order	By the full slot string, so devices group by domain
Where the numbers come from	Which domains exist is Windows' decision. The numbers are the same ones Linux <code>lspci</code> shows on the same VM

IF YOU ONLY EVER TOUCH PHYSICAL MACHINES

You will see `0000` and nothing else, and none of this changes how you use the tool. It matters the first time you run `winlspci` inside a VM with passed-through or SR-IOV devices — where a slot that suddenly has four extra hex digits in front of it is correct, not corrupt.

11 The PCI ID database

Vendor and device names come from `pci.ids`, maintained by the PCI ID Project. `winlspci` bundles a copy so that it works with no network — which, on a lab bench, is usually the situation.

Bundled location	<code>data/pci.ids</code>
Approximate size	~1.4 MB
Source	The PCI ID Project — https://pci-ids.ucw.cz
Licence	BSD / GPL dual-licensed
Network needed to run	No
Date of bundled copy	Shown by <code>lspci -Version</code>

11.1 Refreshing the database

Updates are explicit — the tool never fetches in the background:

```
PS> Update-PciIds           # from https://pci-ids.ucw.cz
```

The update is written so that a bad download cannot leave you worse off than before it ran:

- It downloads to the **temp directory**, not over the live database.
- It **proves the file parses** as a PCI ID database — vendors are present, a known vendor resolves, a known class resolves — before touching anything.
- It keeps the previous copy as `pci.ids.bak`.
- It forces **TLS 1.2** on for older Windows PowerShell hosts, which otherwise negotiate a protocol the server no longer accepts.

A truncated fetch over a captive-portal network would otherwise leave you with a tool that reports numeric IDs and no names, at the exact moment you need names.

11.2 When a name looks wrong

VENDOR ID 11F8 IS MICROCHIP, WHATEVER NAME THE DATABASE SHOWS

Microsemi acquired PMC-Sierra; Microchip acquired Microsemi; the vendor ID never changed. Databases from before mid-2026 still read “**PMC-Sierra Inc.**”, so a Microchip Switchtec switch showed as PMC-Sierra. The bundled copy now says “**Microchip Technology**”.

A test accepts any of the three names, so the answer stays stable whichever vintage of `pci.ids` you are running — and nobody at a bench at 11pm concludes they have the wrong part.

The general lesson holds beyond this one case: a vendor name in `pci.ids` records who registered the ID, not who sells the silicon today. When a name surprises you, check the acquisition history before you check the hardware.

12 Design principles

Three decisions shape how `winspci` behaves in the cases that matter. All three come from the same instinct: **a wrong answer that looks right is worse than an obvious failure.**

12.1 “Not reported” is never rendered as zero

Root ports and chipset devices legitimately expose no link state. Rendering that as `x0` would look like a dead link — a false alarm that costs somebody an hour. So an absent value stays `$null` internally and prints as “*not reported by this device*”. `Present` is a real field in attribute output for the same reason.

The same rule covers two related cases:

Missing class codes

The host bridge reports no class property at all, so the class is read from the `CC_0600` hardware ID instead. A device with no class from either source prints *Unknown class [????]* rather than class `0000`, which would be a real class code the device does not have.

The downtrained flag

`DOWNTRAINED` is only ever said when *both* the current and the maximum are reported. In PowerShell `$null -lt 4` evaluates to true, and an earlier version used that comparison directly — so it flagged a width that was never there.

There are tests for each of these.

12.2 A filter that matches nothing exits non-zero

`lspci -d 11f8:` finding no card must not look like success. That is the difference between “*the card is not there*” and “*the command worked*”, and a build script needs to be able to tell them apart. The same rule applies to `-Downtrained` and to an attribute query that returns no rows.

12.3 Selectors are parsed, not string-matched

`-s` follows lspci's grammar field by field rather than comparing against the raw address string. The consequences are all things that used to be wrong:

- `1:` and `01:` agree, because both are parsed as bus 01 rather than compared as text.
- `-s 1` means **device 01 on any bus**, as it does in lspci. An earlier version read it as bus 01 — the same string, a different question.
- `.0` and `:00.0` work, selecting by function and by device.function.

`-d` compares vendor and device as **hex numbers**, so `-d 8:` no longer quietly returns every Intel device. Anything that is not valid hex produces one clear error, rather than a .NET exception per device.

Absent ≠ zero

Missing data prints as missing, never as a value that looks like a fault.

Empty ≠ success

No match is a non-zero exit, so automation can branch on it correctly.

Selectors are parsed

A slot or ID selector means what it means in lspci, not what string comparison happens to do.

13 Performance

Full enumeration of a laptop (24 devices) takes about 1.7 seconds in-process, or about 2.5 seconds as a fresh `lspci` command — of which roughly 0.4 seconds is PowerShell starting up and importing the module.

~1.7 s

Full enumeration, in-process

~2.5 s

Fresh `lspci` command

~0.6 s

`-ListAttributes`

~30 ms

Per-device property fetch

`-ListAttributes` is quick because it answers from the module's static list rather than enumerating the machine at all.

13.1 How it got there

Reaching those numbers took several attempts. Two of them are worth recording.

APPROACH	TIME
<code>Get-PnpDevice</code> + per-property <code>Get-PnpDeviceProperty</code>	> 5 min
Batched <code>Get-PnpDeviceProperty -KeyName <16 keys></code>	~36 s
<code>Get-PnpDeviceProperty -KeyName '*'</code>	<i>appeared 2× faster — returned zero properties</i>
<code>Win32_PnPEntity (SELECT *) + Invoke-CimMethod GetDeviceProperties</code>	~2 s
<code>SELECT <7 columns> ... WHERE PNPDeviceID LIKE "PCI\VEN[_]%"</code> + the same method	~1.7 s

THE INTERESTING FAILURE

The wildcard looked like the fastest option and was in fact doing nothing, because `-KeyName` does not support wildcards. The benchmark was measuring a no-op, and the output still looked plausible. Only checking the *values* caught it — a reminder that a performance test which does not assert on results is measuring the wrong thing.

13.2 Why the per-device path is fast

The per-device property fetch calls the CIM method that the cmdlet wraps: roughly **30 ms per device versus ~1230 ms**. The cmdlet appears to re-resolve the device on every call, while the CIM method takes an already-resolved instance and works from it.

13.3 Where the rest of the time actually went

The second surprise was that the remaining time was not in the per-device calls at all — it was in the `Win32_PnPEntity` enumeration itself. A `WHERE` clause alone barely moves it; *projecting* the seven columns

the module actually reads halves it, because the provider materialises far less per instance.

THE WQL ESCAPING IS LOAD-BEARING

The `LIKE` text has exactly **two backslashes** and `[_]` for the underscore, because `_` is WQL's single-character wildcard. Get the escaping wrong and the query returns **zero rows silently** — which renders as a machine with no PCI bus at all. A test pins the query's row count against a client-side `-like` so that this cannot regress unnoticed.

Parsing `pci.ids` costs about 50 ms and is not worth optimising.

14 Exit codes and scripting

winlspci is built to be used from scripts as well as by hand, so the exit code carries meaning. There are four of them, and the distinctions between them are deliberate.

CODE	MEANING	WHEN YOU SEE IT
0	OK	The command ran and completed successfully
1	Matched nothing	A filter — <code>-s</code> , <code>-d</code> , <code>-Downtrained</code> , or <code>-Attribute / -Match</code> — matched nothing. The command itself worked; the hardware is simply not there
2	Impossible or not implemented	The request cannot be answered here (<code>-x</code>), or it is a known lspci flag this tool does not implement (<code>-m</code> , <code>-p</code> , <code>-b</code> , ...). An explanation is printed, with a pointer to the nearest equivalent
64	Usage error	An unknown option, or a selector that is not valid hex

14.1 Reading the difference between them

Each code answers a different question, which is the whole point of having four:

1 — about the machine

The command was fine. The hardware you asked about is not present.

2 — about the tool

The command was understood, and this tool will not or cannot answer it.

64 — about the command

The command was not understood at all: a typo, or a selector that is not hex.

14.2 Using it in a script

```
lspci -d 11f8: | Out-Null
switch ($LASTEXITCODE) {
    0 { 'switch present' }
    1 { 'switch absent' }
    2 { 'winlspci cannot answer that here' }
    64 { 'bad command line' }
}
```

The *semantics* match Linux, so a colleague's `lspci -d 11f8: || echo "absent"` translates directly — but write the test against `$LASTEXITCODE` as above rather than with `||`, which Windows PowerShell 5.1 does not support.

STRUCTURED OUTPUT IN AUTOMATION

For anything that another program will parse, prefer `-Json` or `-Csv` over `-Delimited`: both quote correctly and neither depends on a fixed column order staying fixed.

15 Troubleshooting

The situations below follow directly from how the tool works. Most of them are the tool being correct rather than the tool being broken.

SYMPTOM	CAUSE AND RESOLUTION
<code>-s 1</code> returns something unexpected	Not a defect. Following <code>lspci</code> 's grammar, a bare <code>1</code> means device 01 on any bus , not bus 01. For a bus, write <code>1:</code> or <code>01:</code> — those two agree. See §12.3.
<code>-d 8:</code> does not return every Intel device	Correct. Vendor and device are compared as exact hex IDs, so <code>-d 8:</code> is vendor <code>0008</code> . Only the <i>class</i> field is a prefix match. See §5.1.
<code>-s01:</code> or <code>-s .0</code> misbehaves, but only when run from a prompt	You are calling <code>lspci.ps1</code> directly, so PowerShell's parser sees the value first. Use the <code>lspci</code> launcher on PATH, or put a space before the value and quote anything starting with a dot or colon: <code>-s '.0'</code> . See §5.2.
A slot has four extra hex digits in front of it, e.g. <code>7870:00:00.0</code>	Correct, and expected inside a VM. That prefix is the PCI domain (segment), which Hyper-V and Azure pack into the upper bits of Windows' bus number. See chapter 10.
<code>-s 00:00.0</code> returns several devices on a VM	Intended. A selector without a domain matches every domain. Qualify it — <code>-s 7870:00:00.0</code> — to pick one. See §10.2.
A device shows no link speed or width	Not a fault. Root ports and chipset devices legitimately expose no link state; the field is <i>not reported</i> , which is deliberately distinct from a value of zero. See §12.1.

SYMPTOM	CAUSE AND RESOLUTION
A device reports less than its maximum speed or width	Real, but not automatically a problem — link power management routinely lowers both at idle. Use <code>-Downtrained</code> to list every such device, then judge in context.
A device shows <i>Unknown class</i> [????]	Correct behaviour. Windows reported no class property and no <code>CC_xxxx</code> hardware ID for it. Printing class <code>0000</code> instead would name a real class the device does not have. See §12.1.
A vendor name looks wrong (e.g. a Microchip part reads PMC-Sierra)	Correct behaviour. <code>pci.ids</code> records who registered the vendor ID, not who owns the product line today. See §11.2.
<code>lspci -x</code> refuses to run	By design. Configuration-space dumps need a signed kernel-mode driver. Exit code 2. See chapter 2.
A known <code>lspci</code> flag exits 2 with “not implemented”	Intended. Recognised-but-unbuilt flags (<code>-b</code> , <code>-m</code> , <code>-p</code> , <code>-P</code> , <code>-i</code> , <code>-q</code> , <code>-M</code> , the access-method flags) refuse rather than let the parameter binder guess at a different command. The message names the nearest equivalent. See §5.4.
A command exits 64	Usage error: the option is not recognised at all, or a selector is not valid hex. Distinct from exit 2, which means the option <i>was</i> recognised. See chapter 14.
<code>-D</code> and <code>-d</code> behave differently	They should. Short <code>lspci</code> flags are case-sensitive: <code>-D</code> is the domain prefix, <code>-d</code> is the ID filter. Long options such as <code>-Json</code> are case-insensitive and may be abbreviated. See §5.1.
A filter returns nothing and the script treats it as failure	Intended: no match is exit 1, not exit 0. Branch on <code>\$LASTEXITCODE</code> rather than on empty output. See chapter 14.
<code>lspci</code> is not recognised as a command	The <code>bin</code> directory is not on PATH for this session. Re-run the PATH line from §3.2, or import the module directly per §3.4.
PATH additions vanished, or the environment broke after setting PATH	Almost certainly <code>setx</code> , which truncates PATH at 1024 characters without warning. Use the .NET API shown in §3.2.
Devices show numeric IDs but no names	The <code>pci.ids</code> database is missing or damaged. Run <code>Update-PciIds</code> , which validates the download before replacing anything and keeps the previous copy as <code>pci.ids.bak</code> . Check the bundled date with <code>lspci -Version</code> .
A parsed one-liner produces the wrong column	The delimiter is stripped from values, not escaped, so column count is fixed — but an <i>empty</i> field is legitimate and means “not reported”. If your parser collapses empty fields, use <code>-Csv</code> instead. See §8.3.

16 Tests

The test suite runs from a stock Windows install with nothing added.

```
PS> powershell -ExecutionPolicy Bypass -File tests\Invoke-Tests.ps1
```

The same suite runs on every push under Windows PowerShell 5.1 on a GitHub `windows-latest` runner (`.github/workflows/tests.yml`) — a different machine with a different PCI inventory, which is the point. Chapter 10 exists because that runner found something no physical bench machine would have.

16.1 Why not Pester

No Pester, deliberately. Stock Windows ships Pester 3.4.0, whose `Should Be` syntax is incompatible with Pester 5's `Should -Be`. A Pester suite would therefore pass on the author's machine and fail on a colleague's — precisely the class of misleading result the rest of the tool is built to avoid.

16.2 Hardware-dependent tests

The PCI inventory differs on every machine, so hardware-dependent tests assert on *shape* rather than on values. Where a test genuinely needs a specific device present, it skips with a stated reason rather than failing or, worse, passing vacuously.

WHAT THE SUITE PINS

Among others: that absent values never render as zero (§12.1); that `1:` and `01:` resolve identically (§12.3); that every enumerated device appears in the topology tree exactly once (§9.2); and that vendor `11f8` resolves to any of its three registered names (§11.2).

A Appendix A — Flag quick reference

One page to keep next to the bench. Short lspci flags are case-sensitive and combine (`-tv`, `-nnk`, `-vvnn`); long options are case-insensitive and may be abbreviated to a unique prefix.

Selection

<code>-s <selector></code>	Select by slot, using lspci's grammar with every field optional: <code>01:</code> (bus), <code>01:00.0</code> , <code>1</code> (device 01 on any bus), <code>.0</code> (every function 0), <code>:00.0</code> , <code>0000:01:00.0</code> . Padded or unpadded
<code>-d [ven]:[dev][:class]</code>	Select by ID. Vendor and device are exact hex; class is a prefix
<code>-Downtrained</code>	Only devices below their maximum speed or width
<code>-s <dom>:<bus>:...</code>	A selector without a domain matches every domain; one with a domain matches only that domain

Verbosity and format

<code>-n / -nn</code>	IDs only / names and IDs
<code>-v / -vv / -vvv</code>	Link state, driver, status / adds MPS, MRRS, subsystem, NUMA, AER / everything, plus what is missing
<code>-k</code>	Driver and version (the same lines <code>-v</code> shows)
<code>-t</code>	Topology tree with inline link state
<code>-D</code>	Domain on every line (<code>0000</code> normally; a real segment on Hyper-V / Azure). Note: <code>-D</code> $\neq$ <code>-d</code>
<code>-Json / -Csv</code>	Structured output
<code>-Delimited</code>	<code> </code> -separated records
<code>-Delimiter / -Header</code>	Change the separator / name the columns
<code>-Version</code>	Tool version and bundled <code>pci.ids</code> date

Attribute queries

<code>-Attribute <names></code>	One record per attribute; wildcards accepted
<code>-Match <regex></code>	Filter by value
<code>-PresentOnly</code>	Drop attributes the device does not report
<code>-ListAttributes</code>	List what is queryable (static list; does not enumerate)

Cmdlets

<code>Get-PciDevice</code>	Enumerate devices as objects
<code>Format-Lspci</code>	Render device objects in <code>lspci</code> style; takes <code>-Verbosity</code>
<code>ConvertTo-PciAttributeRecord</code>	Expand device objects to one record per attribute
<code>Update-PciIds</code>	Refresh the bundled <code>pci.ids</code> database

Refused / not implemented

<code>-x, -xxx, -xxxx</code>	Refused — needs a signed kernel-mode driver; exits 2
<code>-b, -m, -mm, -p, -P, -i, -q, -Q, -A, -O, -F, -G, -H1, -H2, -M</code>	Recognised but not implemented; exits 2 with a pointer to the nearest equivalent

Exit codes

0	OK
1	A filter matched nothing
2	Impossible here, or a known flag not implemented
64	Usage error: unknown option, or a selector that is not hex

B Appendix B — Repository layout

Where things live, for anyone reading or extending the source.

winlspci.psd1	manifest (exports, version, release notes)
winlspci.psm1	loader: module state, then dot-sources Private\ and Public\
Public\	exported functions, one file per function or cohesive group
Get-PciDevice.ps1	enumeration and the device object shape
Format-Lspci.ps1	lspci-style text
Format-PciTree.ps1	-t
Attributes.ps1	ConvertTo-PciAttributeRecord, Get-PciAttributeName
Format-PciDelimited.ps1	-Delimited
PciIds.ps1	pci.ids parse, lookups, Update-PciIds
Private\	internal helpers
DeviceProperties.ps1	DEVPKEY fetch, BDF, class-from-hardware-id
Selectors.ps1	-s and -d parsing
Helpers.ps1	downtrain test, StrictMode-safe field access, text sanitising
bin\lspci.cmd / .ps1	the command-line front end and its argument parser
data\pci.ids	bundled PCI ID database
tests\Invoke-Tests.ps1	the suite (no Pester)

Reading it against this manual

Private\Selectors.ps1	The <code>-s</code> and <code>-d</code> grammar described in §5.1 and §12.3
Private\DeviceProperties.ps1	The DEVPKEY fetch behind chapter 13's performance work, the BDF assembly behind chapter 10, and the class-from-hardware-ID fallback in §12.1
Private\Helpers.ps1	The downtrain test guarded in §12.1, and the text sanitising that strips the delimiter from values (§8.3)
Public\PciIds.ps1	Everything in chapter 11, including <code>Update-PciIds</code>
bin\lspci.cmd / .ps1	The launcher and argument parser discussed in §5.2 — the reason the <code>.cmd</code> front end sees your command line before PowerShell's parser does

C Appendix C — Glossary

AER	Advanced Error Reporting. A PCI Express capability for logging and reporting link and transaction errors. winlspci reports whether the capability is <i>present</i> ; the register contents require a live read and are out of reach.
BDF	<code>bus:device.function</code> — the address that identifies a PCI function. Written <code>01:00.0</code> , optionally prefixed with a domain: <code>0000:01:00.0</code> .
Configuration space	The per-function register space that holds PCI identity and capability structures. Reading it directly from userland is not possible on Windows — the reason winlspci works the way it does.
Domain / segment	The highest-order part of a PCI address. <code>0000</code> on an ordinary machine; Hyper-V and Azure report a real segment, packed into the upper bits of Windows' bus number. A non-zero domain is always shown in the slot. See chapter 10.
Downtrained	A link operating below its maximum speed, width, or both. Often power management rather than a fault.
GT/s	Gigatransfers per second — the PCIe signalling rate. 8GT/s is Gen3, 16GT/s Gen4, 32GT/s Gen5, 64GT/s Gen6.
Link width	The number of lanes in use, written x1, x4, x8, x16. Reported both as negotiated and as maximum.
MPS	Max Payload Size — the largest data payload the device is configured to move in one transaction.
MRRS	Max Read Request size — the largest read the device will issue in a single request.
NUMA node	The memory/CPU locality domain a device is attached to. Relevant to bandwidth and latency on multi-socket systems.
pci.ids	The community-maintained database mapping numeric PCI IDs to vendor and device names. Bundled with winlspci; refreshed with Update-PciIds .
PnP	Plug and Play — the Windows subsystem that enumerates devices. winlspci reads its data, populated by the PCI bus driver from configuration space at enumeration time.
Prog-if	Programming interface — the third component of the PCI class code, after class and subclass.
Root port	The PCIe port on the host side of a link. Frequently reports no link state of its own, which winlspci renders as <i>not reported</i> rather than zero.
Subsystem ID	Vendor and device IDs assigned by the board integrator, distinguishing one implementation of a chip from another.

D Appendix D — Licence and credits

Licence

winlspci is released under the **MIT Licence**.

The bundled PCI ID database, `data/pci.ids`, is **BSD / GPL dual-licensed** and comes from the PCI ID Project. It is redistributed under those terms and is not covered by the MIT licence above.

Credits

pci.ids	Maintained by the PCI ID Project — https://pci-ids.ucw.cz
lspci	The original Linux utility, part of the pciutils package, whose interface winlspci deliberately follows
Source and issues	https://github.com/jpmutschler/winlspci

Scope of this document

ABOUT THIS MANUAL

This manual documents the behaviour of winlspci as released. Chapter 15 and Appendix C organise and expand on behaviour described elsewhere in the documentation set; they introduce no flags, cmdlets or output formats beyond those listed in chapters 5 and 6. Where the tool's capabilities are bounded by Windows itself, chapter 2 is the authoritative statement of that boundary.



WINLSPCI — POWERSHELL PCI TOOL (NO DRIVER)

Document revision 1.2 · August 2026 · Specifications subject to change